

# Designing a Domain-Driven Authentication Service with .NET 10, Dapper, and JWT

## Introduction

Building a reusable authentication service requires careful **architecture and design**. In this guide, we focus on creating a **back-end REST API** for authentication, using **.NET (latest)** with **Dapper** for data access and **JWT** for token-based authentication. We will apply **Domain-Driven Design (DDD)** principles strictly – defining clear domain models (Users, Roles, Permissions, etc.), using **Aggregates** and **Value Objects**, and adhering to **SOLID** best practices. The goal is a flexible “User Management” service (think of it as an **Identity and Access Management** microservice) that can be a boilerplate for multiple projects. We’ll limit initial implementation to classic username/email + password login (with JWT issuance), but design the system to be **extensible** (e.g. adding external providers or more features later). No external auth frameworks (like ASP.NET Identity) will be used – instead, we’ll implement the logic ourselves for learning purposes. This document provides a comprehensive architecture roadmap that you can follow to implement the authentication service.

## Domain-Driven Design Principles for Authentication

**Domain-Driven Design** centers your software around the core domain and its logic. In our case, the domain is **Authentication and User Identity Management**. We treat this as its own **bounded context** – all user accounts, credentials, roles, permissions, and related behaviors live in this context, separate from any other application domain (e.g. separate from a content or business domain). The **ubiquitous language** of this domain includes terms like “User”, “Role”, “Permission”, “Login”, “JWT Token”, “Group”, etc., which we will define clearly. By modeling these concepts explicitly, we ensure the code reflects real authentication scenarios and rules.

Key DDD patterns we will use:

- **Entities:** Objects with identity and lifecycle, e.g. a User or Role (identified by an ID).
- **Value Objects:** Immutable value types that represent concepts with no identity (e.g. an Email address or a Password hash).
- **Aggregates:** Clusters of domain objects bound by a root entity. The aggregate root (e.g. User) guarantees consistency of changes within the aggregate.
- **Domain Services:** Domain operations that don’t naturally fit as a responsibility of a single Entity or Value Object. For example, generating a JWT token or verifying a password might be done by a domain service if it involves multiple parts of the domain.
- **Repositories:** Abstractions for persisting and reconstituting aggregates. Repositories mediate between the domain and the data store (here, Dapper/SQL Server), so that domain logic is decoupled from data access concerns.

**Bounded Context:** We have an “Identity and Access Management” context. Other parts of a larger system (e.g. an Orders context) would not directly access our user tables, but instead use our context’s API. This isolation follows DDD’s strategic design – it keeps the auth domain cohesive. For instance, if another service needs to check user roles, it would call our Authentication service’s API rather than query its database directly. This context boundary prevents leakage of auth logic into other domains <sup>1</sup> <sup>2</sup> .

By following DDD, we aim for a design where **business rules of authentication** are paramount, and technical details (JWT, Dapper, etc.) are kept in infrastructure layers. We’ll also ensure **Clean Architecture** layering: the domain model and business logic form the core, and implementation details (DB, JWT library) depend on the core, not vice versa <sup>3</sup> . This makes our service flexible: for example, we could swap SQL Server for another database without changing core domain logic, or reuse the domain layer in a different host (monolith, microservice, etc.) <sup>4</sup> <sup>5</sup> .

## Domain Model: Entities and Value Objects

Let’s define the core domain model for the authentication service. These are the **Entities** with their attributes and the relationships between them (essentially our domain’s data model). We will also identify some attributes that deserve to be modeled as **Value Objects** for better encapsulation and validation.

### Primary Entities (Aggregates):

- **User** – Represents an individual user account in the system. Key fields:
  - `UserId` (unique identifier for the user – likely a GUID or int).
  - `Username` and/or `Email` (unique login identifiers).
  - `PasswordHash` (hashed password) and `PasswordSalt` (for hashing, if using salted hashes).
  - `Status` (e.g. Active, Locked) and `CreatedOn` timestamp.
- Relationships: A User can have multiple Roles and can belong to multiple Groups. Also, a User may have many Sessions (active JWT tokens), PasswordResetTokens, etc.
- **Behavior:** User entity will contain logic for things like verifying a provided password, updating password, checking if account is active/locked, etc. It may also have methods to manage roles or groups (add/remove role assignments) or to create tokens (perhaps through domain services).
- **Aggregate role: User is a likely aggregate root** – it’s the central entity through which much of the auth operations occur. Many related objects (roles, groups, sessions) tie back to a user.
- **Role** – Represents a user role (e.g. “Admin”, “User”, “Guest”). Fields:
  - `RoleId` (PK), `Name` (unique role name), `Description` .
- Roles could be used for coarse-grained access control (e.g. an “Admin” role might imply certain privileges).
- Relationship: Many-to-many with Users (one user can have multiple roles, and each role can be assigned to many users). This is typically implemented via a join table (UserRole).
- **Aggregate role:** A Role could be an aggregate root if we manage roles independently (e.g. an admin can create new roles or assign permissions to roles). In our design, Roles are relatively static and simple (no complex internal structure beyond name/description), so they might be managed via a simple repository or even as reference data.

- **Permission** – Represents a fine-grained permission or privilege (e.g. “CanResetPasswords”, “ViewReports”). Fields:
  - `PermissionId` (PK), `Name` (unique permission key), `Description`.
  - Relationship: Permissions are not directly assigned to users in our model; instead they are assigned to Groups (or potentially to Roles if extended).
  - *Aggregate role*: Likely a simple entity. Often, permissions are treated as value objects (if they’re fixed code enums) or as a reference entity. We will consider it an entity since it’s many-to-many with groups.
- **Group** – Represents a collection of users (for collective permission assignment). For example, a group might represent a department or a project team, where you assign certain permissions to that group, and all members inherit them. Fields:
  - `GroupId` (PK), `GroupName`, `Description`.
  - Relationship: Many-to-many with Users (a user can be in multiple groups, a group can have multiple users) via `UserGroup` link. Also many-to-many with Permission via `GroupPermission` link (a group can have many permissions, and a permission can belong to many groups).
  - *Aggregate role*: Group can be an aggregate root that encompasses its permission assignments. For instance, the Group aggregate might contain a collection of Permission IDs/objects that are granted to the group. Managing a group’s permissions (add/remove permission) would be domain behavior inside the Group aggregate. Users are not contained within Group aggregate (they are separate aggregates), but a Group holds references (IDs) of users or just leaves the linking to the link table managed by repositories.

### Supporting Entities:

- **UserRole** – Join entity linking Users and Roles (to realize the many-to-many). Fields: composite key of `UserId` + `RoleId`. In code, we might not need a full entity class for this link; it can be managed via collections on User (like a `List<RoleId>` or `List<Role>` within the User aggregate, or a mapping in the repository). But we can also model it explicitly if needed for certain operations.
- **UserGroup** – Join entity linking Users and Groups. Fields: `UserId` + `GroupId` composite key. Similar to UserRole, this can be seen as an internal relationship managed by the User and Group aggregates via repository.
- **GroupPermission** – Join linking Groups and Permissions. Fields: `GroupId` + `PermissionId`. This represents assignment of specific permissions to a group. In the domain, this could be represented by a collection of Permission within Group (or a mapping object).

*(Note: We do not have a direct RolePermission in our initial design – roles are more for categorization, while permissions are granted via groups. This approach is one way to handle flexibility: you can assign fine-grained*

permissions through groups, and use roles for broader user types. If needed, a RolePermission linking table could be added later to assign permissions directly to roles as well.)

- **Session** – Represents an active authenticated session or token issued to a user (for JWT, this could correspond to a refresh token or a JWT that we track). Fields:
  - `SessionId` (PK), `UserId` (owner of session), `JwtToken` (the JWT string or a token identifier), `IssuedAt` timestamp, `ExpiresAt` timestamp.
  - This is useful if we want to track active tokens (so a user can have multiple concurrent sessions, and we can implement logout or token revocation by invalidating a session record). If we go fully stateless JWT without server tracking, we might not persist sessions; but for security (e.g. token blacklist or user logging out), having a Session store is helpful.
  - Relationship: Many-to-One with User (each Session is for one User; a User can have many Sessions).
- *Aggregate role:* Session could be a separate aggregate (managed by a SessionRepository). However, one might treat Session as a subordinate of User (since it belongs to a user). For simplicity, we can handle sessions via a repository that references user ID – no need to load all sessions with a user unless needed.
- **PasswordResetToken** – Represents a one-time token for password recovery workflow. Fields:
  - `TokenId` (PK or could reuse the token string as PK), `UserId`, `Token` (secure random string), `ExpiresAt` timestamp.
  - Used when a user requests a password reset – we create one, store it, email it to the user. When the user presents it with a new password, we verify it and allow password change.
  - Relationship: Belongs to a User (User can have multiple password reset tokens issued over time). Typically, only the latest one might be valid if you choose to invalidate old ones.
- *Aggregate role:* Could be independent (look up by token string), or treated as part of the User aggregate (but storing potentially many tokens on User isn't ideal). We will likely use a repository for tokens for quick lookup.
- **EmailVerificationToken** – Similar structure to PasswordResetToken, but for verifying a user's email address upon registration. Fields: `TokenId`, `UserId`, `Token`, `ExpiresAt`. Also one-to-one or many-to-one with User. This is used after a new user registers – they get an email with a verification token link; when they use it, we confirm their email and possibly activate their account.
- **AuthenticationProvider** – Represents an external auth provider link (for future extension, e.g. OAuth login with Google, or enterprise LDAP). Fields:
  - `ProviderId` (PK), `UserId` (the user account it's linked to), `ProviderType` (e.g. "Google", "Facebook", "LDAP"), and `ProviderUserId` or identifier (e.g. the Google account ID).
  - Relationship: Each entry attaches one external account to one internal User. A User can have many AuthenticationProviders (linking multiple external logins).
- *Aggregate role:* Likely part of the User aggregate (as a collection of external logins), since these are tightly coupled to user identity.
- **AuditLog** – Represents an audit trail of important events (for security monitoring). Fields:

- **LogId** (PK), **UserId** (who performed or was subject of the action), **EventType** (e.g. "LOGIN\_SUCCESS", "LOGIN\_FAILURE", "PASSWORD\_CHANGED", etc.), **Timestamp**, **Metadata** (like IP address, user-agent, etc.).
- Relationship: Many-to-One with User (user has many audit log entries). Could also log events with null user (system events).
- *Aggregate role*: Treat this as a separate store for logging. Typically we wouldn't load logs as part of User aggregate; instead, logs are appended as side-effects of actions.

**Value Objects:** In DDD, we use *value objects* to encapsulate concepts that have value equality and business rules, and to avoid “primitive obsession” (using raw strings/integers for things like emails, passwords, etc., without behavior). In our auth domain, good candidates include:

- **EmailAddress** – A value object for a user's email. It can enforce format validation (for example, ensuring it matches a valid email regex or RFC standard) upon creation. Instead of storing email as a plain string everywhere, an EmailAddress VO ensures any email in our system is valid by construction. For instance, the Email VO can validate format according to RFC 2822 when created <sup>6</sup> <sup>7</sup>. It's immutable; if you want to change a user's email, you create a new Email VO after validating.
- **Password** – This could be modeled in a couple of ways. We do **not** want to ever hold a plaintext password in our system except momentarily for checking. We might define:
  - **RawPassword** VO: used transiently when a user is registering or logging in, to validate it meets complexity rules (length, characters, etc.) <sup>6</sup> <sup>7</sup>. This VO would ensure a password is, say, 8+ characters and maybe includes required character types. We would then hash it and discard the plaintext.
  - **PasswordHash** VO: encapsulates the hashed password (and possibly the salt). This VO can ensure the hash string is of the expected format/length. It could also contain a method to verify a given RawPassword against the hash (using the appropriate hashing algorithm and salt). By making it a VO, we bundle the hash and the logic of verification together. For example, the PasswordHash VO might contain a method `Verify(rawPassword)` that hashes the input and compares it to itself.
- Alternatively, instead of two VOs, we could have one **Password** VO that either holds a plaintext (for new passwords) or a hash, and can produce a hashed version via a method. But it's cleaner to keep them separate or at least to ensure that after user creation, only the hashed form is kept around.
- **JWT Token** – The JWT string itself might be seen as just a string, but we could wrap it in a VO for clarity. A **JwtToken** VO might encapsulate the token string and maybe provide some parsing or validation capability (like checking expiry or decoding claims for internal use). However, often JWT handling is done by libraries and seen as a technical detail. We might treat it simply as a string in the domain and let infrastructure handle generation/validation. If we plan to support token revocation, we use the **Session** entity to track them.
- **UserStatus** – We might make an enum-like VO for user status (Active/Locked/Disabled) to make it typesafe and include any rules (e.g. a method `IsActive`).

- Other possible VOs: **Username** (if we want to enforce certain format or length rules on usernames), **GroupName** similarly, or even **RoleName** if needed. These add rigor by validating invariants at creation.

Using value objects for things like Email and Password helps keep our **domain model consistent and valid**. For example, we won't accidentally store an invalid email, because it wouldn't construct an Email VO. The cited approach in one DDD example is that the User aggregate uses Email and Password as VOs that encapsulate validation logic, ensuring emails are well-formed and passwords meet security criteria <sup>6</sup> <sup>8</sup> .

## Aggregates and Relationships

With the domain entities outlined, we need to organize them into **Aggregates**. An aggregate is a cluster of related objects that are treated as a unit for data changes – it has a single **aggregate root** entity that external code interacts with. We enforce invariants within aggregates and maintain consistency through the aggregate root.

**User as an Aggregate Root:** The **User aggregate** is central. It encompasses the User entity and closely related value objects and entities. We can include within the User aggregate:

- The user's core data (username, email VO, password hash VO, etc.).
- The user's **Roles**: perhaps as a collection of Role identifiers or a small object representing the assignment. When adding/removing a role, we do it through the User aggregate (e.g., `user.AssignRole(role)` method) which ensures business rules (maybe you can't assign duplicate roles, etc.). The actual Role entity may be in a different aggregate (if roles are managed independently), so User might just store role IDs or a `List<RoleAssignment>` VO. We might load the Role's details when needed via repository.
- The user's **Group memberships**: similarly, the User aggregate could track groups it belongs to via IDs or lightweight references. However, since Group is another aggregate, we wouldn't keep full Group entities inside User; instead perhaps a collection of `GroupId` value objects or a small `UserGroupMembership` VO. Business logic, like ensuring a user isn't added to the same group twice, can be handled in the method that adds a group to the user.
- The user's **AuthenticationProviders** (external logins): these could be contained within the User aggregate as a collection of VO/entity (since they are only used in context of the user). For instance, a method `user.AddExternalLogin(providerType, providerId)` could add a new AuthenticationProvider entry (VO) to the user. This keeps external account links within the user's consistency boundary.
- Possibly, the user's **Sessions**: One approach is to include active sessions in the user aggregate (e.g., a collection of Session entity instances that belong to that user). If we do that, whenever we add or remove sessions (like on logout), we'd load the user aggregate and modify its Sessions collection. This guarantees that, for example, you could enforce a rule "a user can have at most N active sessions" by checking the collection before adding a new one. However, if users can have many sessions over time, loading them with the user every time could be heavy – so you might choose not to aggregate sessions under user. An alternative is to treat Session as a separate aggregate (with just a reference to user) and manage it via a Session repository. In practice, we often treat session/token store separately for performance. We will lean towards keeping Session as a separate repository, but ensure that when a user is deleted or deactivated, we also clear their sessions (this can be done via domain events or application logic).

**Group as an Aggregate Root:** The **Group aggregate** would include the Group entity and its assigned permissions. A Group has many Permissions (via GroupPermission links). We can model that as a collection of Permission IDs or Permission value objects inside the Group aggregate. For example, Group might have

a method `group.AssignPermission(permission)` which adds a permission to its collection, and ensures no duplicates. This way, the Group aggregate encapsulates the invariant that “permissions in a group are unique and valid”. The actual Permission entities could be simple references (just an ID and name) or we load permission details on demand. The **Group** might also have methods like `AddUser(userId)` or `RemoveUser(userId)` to manage membership. But adding a user to a group might equally be done from the User side or via an application service – we need to decide who “owns” that relationship. In DDD, if an association is truly many-to-many and both sides are important, you sometimes introduce a domain service to handle it. Or pick one side to handle consistency. It might be simpler to handle user-group membership from the User aggregate (since one user joining or leaving a group is an action concerning that user), while the Group aggregate mostly manages its permissions. Alternatively, we treat Group as owning the relationship and have a collection of user IDs in Group as well. To avoid duplication, an approach is: when a user joins a group, call `group.AddMember(userId)` which could internally add to a members list and also perhaps notify the User aggregate (though cross-aggregate communication would be via a domain event or application layer). To keep it simple, we can manage membership on the User side for now, and just ensure referential integrity via checks in repositories.

**Role as an Aggregate (or not):** Roles in our design are fairly static – you might have a predefined set or allow an admin to create roles. A Role doesn’t have other entities within it, except maybe a list of associated permissions if we extend it. For now, Role could be an independent small aggregate (just itself). Users reference roles by ID. Managing which roles exist and maybe assigning some semantic (like admin role implies certain group memberships) could be handled outside the domain or via policy. We won’t complicate roles with their own permissions here (since we have groups for that), but note that the design could easily be extended to have `RolePermission` similar to `GroupPermission` if needed.

**Permission as an Entity:** Permission is typically reference data. We could treat it as part of Group aggregate (i.e., the Group holds Permission IDs; you load the Permission details from a repository if needed for display). If we need to manage permissions (add new permission types, etc.), they have their own repository.

#### Other Entities:

- **PasswordResetToken** and **EmailVerificationToken** are likely separate aggregates or at least separate entities not contained within User. Why? Because when a user submits a token (e.g. they click the link from email with token), we need to find which user it belongs to quickly. That means querying by the token value. If these were inside User aggregate, we’d have to scan all users or all tokens – not efficient. Instead, we have a repository (or just a table) we can query by token. The token entity can carry the `UserId` and expiry. The logic to use it (like resetting password) will involve both the token repository and the user repository (find token -> get user -> allow password change -> delete token). This coordination can be done in an application service, or via a domain service that loads the necessary aggregates.

- **AuditLog** entries will be generated as side-effects of domain events (like on successful login, on password change, etc.). We treat AuditLog as write-only log store (maybe an **event stream** or just table). It’s separate from the main aggregates; typically you wouldn’t roll back a user change if writing an audit log fails – you’d rather log best-effort. So this is more of an eventual consistency/out-of-band thing. We might implement it by raising a domain event (e.g. `UserLoggedInEvent`) that triggers an `AuditLog` record to be saved, or simply call an audit logging service method in the application layer after the core action.

By delineating aggregates, we follow the rule that **one transaction should modify only one aggregate** (to maintain consistency and avoid complex two-phase commits etc., especially if this became a distributed system). For example, when registering a user, we will create a new User aggregate (User entity plus maybe an EmailVerificationToken entity). We can persist the User and the Token either in the same database transaction (since it's the same context, that's acceptable), but in pure DDD each is its own aggregate. Often in such cases, you save the User (aggregate1) and save the Token (aggregate2) in one go as part of the use-case, which is fine as long as you handle consistency (or you could treat the token creation as a domain event reaction). For simplicity, we can handle them together in the application service transaction.

### Example Aggregate Structure:

- **User Aggregate:** User (root) – value objects: Email, PasswordHash, etc. – possibly child entities: ExternalAuthProviders. Collections of role IDs and group IDs (as references). Methods like `Register(...)`, `ChangePassword(...)`, `Login(password)`, `Lock()`, `Unlock()`, `AddRole(roleId)`, `RemoveRole(roleId)`, etc. All modifications to user's state go through these methods, which enforce rules (for example, `ChangePassword` might require verifying the old password or ensuring the new password meets criteria).
- **Group Aggregate:** Group (root) – possibly a collection of Permission IDs as part of it. Methods: `AddPermission(permId)`, `RemovePermission(permId)` to manage its permissions, with checks. Possibly `RenameGroup(newName)` etc. (with validation). If we decide Group also manages membership: `AddMember(userId)` could ensure no duplicates. But adding a member might also require ensuring the user exists (which might be done via repository check or domain service).

### Aggregate Invariants and Rules:

Each aggregate enforces the business rules relevant to it. Some examples of rules in our auth context:

- A **User** must have a unique username and email. This is a rule that spans aggregates (no two User aggregates can have the same email). Enforcing uniqueness usually requires a database check. Our design: when registering or changing email, the application service or repository will check for conflicts (e.g. `UserRepository.FindByEmail(email)` returns null to proceed). This kind of invariant might be enforced via a domain service or repository check because it's a global consistency rule.
- A User's **password** must be stored securely (only hashed). We ensure that by design: the User aggregate will never hold plaintext after construction. Either the factory method to create a User will hash the password, or the Password VO will produce a hash. So at rest, we only have PasswordHash.
- **Email verification:** A new user might have a flag `IsEmailVerified` default false until they click the token. We enforce that unverified users may have limited access. The domain model can expose something like `user.IsVerified` property. The logic that prevents login for unverified users (if we choose to enforce that) could be in the domain (e.g., `User.Login()` could throw an error if not verified), or in application layer. This is a design choice; often verification is handled at application level (allow login but restrict actions, or disallow login until verified depending on policy). We will consider requiring verification for activation.
- **Account lockout:** If a user is locked (Status = Locked), the domain should prevent authentication. E.g., `User.VerifyPassword()` might first check `Status` and fail if locked. We might also implement incrementing failed login attempts and auto-locking after certain attempts as part of domain logic (with fields for FailedAttempts, LockoutUntil, etc., not in initial model but could be added).
- **JWT expiration:** The JWT token has its own expiry encoded. If we store Session with an expiry, we have a second layer of tracking. Ensuring a session is valid means checking current time vs. session expiry (the domain service that validates tokens will consider that).
- **Role/Permission consistency:** If roles or permissions are removed or renamed, those changes should



reflect for users and groups. This would be handled via application logic (for example, if an admin deletes a Role, you'd have to remove it from all users; similarly for a Permission, remove from all groups). Those are maintenance operations to consider adding in admin use-cases.

- **Group/Role naming uniqueness:** likely ensure no two roles with same name, no two groups with same name (repository or domain can enforce by checks).

By defining aggregates clearly, we know which parts of the model can be loaded and modified together. For instance, updating a user's profile info and their roles might involve two aggregates (User and Role or just User if we treat roles as IDs). But assigning a permission to a group involves just the Group aggregate (plus verifying the permission exists). This helps in designing transactions and consistency boundaries.

## Domain Services and Authentication Logic

While our entities and value objects cover a lot of the domain behavior, some operations span multiple aggregates or involve external processes. This is where **domain services** or **application services** come in. In DDD, a *domain service* is used when a piece of logic doesn't naturally belong in a single entity (it might involve coordination between them or use external resources like cryptography or sending email). An *application service* (or use-case service) sits in the application layer orchestrating a use-case by calling repositories and domain methods.

For our authentication domain, key operations include: **registering a user, logging in (producing JWT), verifying email, resetting password, assigning roles/groups**, etc. We will outline how to handle some of these with services:

- **Password Hashing Service:** Storing passwords securely is critical. We will use a strong hashing algorithm (e.g. bcrypt or PBKDF2). This can be implemented as a domain service (since choosing how to hash passwords can be considered a domain concern in an auth context, as it relates to security policy). However, it could also be seen as an infrastructure detail. We'll likely create an interface like `IPasswordHasher` with methods `Hash(password)` and `Verify(password, hash)`. The implementation can use a library or custom code (e.g. BCrypt). The domain layer (or application layer) uses this service whenever a password needs hashing or verification. For example, when creating a new User, the application service might call `hasher.Hash(rawPassword)` to get a hash and salt, then pass that into the User entity. Or, we could let the User entity accept a raw password and internally call a domain service to hash it – but injecting services into entities is awkward, so it's cleaner to do it in the application layer or a domain service function that then creates the User. The key is that **no plaintext password goes into the database** and the hashing logic is centralized (Single Responsibility: the hasher does the hashing, the User just stores the hash).
- **JWT Generation Service:** Generating a JWT token from user info is a task likely handled by a library (e.g. `System.IdentityModel.Tokens.Jwt` in .NET). We consider this an **infrastructure service** since JWT is a technical format. However, from a design perspective, we can wrap it in an interface (e.g. `ITokenService` with `CreateToken(User)` and `ValidateToken(token)` methods) and provide a JWT-based implementation. The domain (or application) calls `CreateToken(user)` after successful login to get a JWT string. The reason to abstract it is for flexibility (maybe you switch to a different token method or need to mock it in tests). The token service will likely need some

configuration (signing key, expiration times, etc.), which we can supply from config. The domain doesn't need to know the details, only that it gets a token string (or object).

- **Authentication (Login) Service:** The process of verifying credentials and issuing a JWT touches multiple parts: you need to retrieve the user (User repository), verify the password (using the hasher and the User's stored hash), and if valid, create a Session record and generate a JWT. This coordination can be done in an application service method (e.g. `AuthService.Login(username, password)`). That method will:

- Use `UserRepository` to find the user by username or email.
- If not found or user is not active, reject (return error). If email not verified (depending on policy), maybe reject or warn.
- If found, call `PasswordHasher.Verify(inputPassword, user.PasswordHash)`. Or if `PasswordHash` is a VO with verify method, call `user.PasswordHash.Verify(inputPassword)`.
- If verification fails, record a login failure (maybe increment a counter, add `AuditLog` for failure). If verification succeeds, proceed.
- Create a JWT via Token service: e.g. `token = jwtService.CreateToken(user)` – typically this will embed the user's ID and maybe roles/claims in the token. The JWT payload might include user ID, username, role names or permission claims, and an expiration (e.g. 15 minutes).
- Create a Session entity: e.g. `SessionRepository.Add(new Session(userId, token, expiresAt))`. We might not store the entire JWT (since it can be verified by itself), but storing it (or a token identifier) allows us to manage logouts/blacklist. Alternatively, we could store only refresh tokens in a database if we implement refresh logic. For now, assume we store each JWT with expiry so user can log out.
- Return the JWT to the caller (to be returned in the API response). Also possibly raise a domain event or call `AuditLog` to record a successful login event.

This **AuthService** can be seen as an **Application Service** (since it uses repositories and domain logic to fulfill a use-case). In some literature, it might also be called a domain service if we consider authentication as part of the domain logic. The key is it orchestrates multiple steps. We keep this logic outside of the User entity because it involves token generation (external) and crossing aggregate boundaries (User and Session). Keeping it in a service adheres to SRP (User entity focuses on user state, the service handles the process). This approach is echoed in an example where an `AuthModule` (service) handles login, including password check and token generation, working closely with User aggregate but separate from it <sup>9</sup>.

- **Registration Service:** Similarly, registering a new user might be handled by a `RegistrationService` (or just part of `AuthService`). Steps:
  - Validate the provided data (username, email format, password strength). Some of this validation is handled by value objects (Email VO will only be created if format is valid; Password VO ensures length, etc.). Additional checks like password complexity or ensuring email uniqueness occur here.
  - Check that username/email aren't taken: use `UserRepository.FindByEmail` or `FindByUsername`. If exists, throw an error.
  - Create a new User aggregate instance. Possibly we have a factory or static method: e.g. `user = User.Register(username, EmailAddress(email), RawPassword(password), etc.)`. Inside that, or just after, hash the password. We might do: `hash = passwordHasher.Hash(password)`, then `user = new User(username, EmailVO(email), hash, salt, etc.)`. The User constructor sets default status (maybe Inactive until email verified) and other defaults.

- Save the new user via `UserRepository.Add(user)`.
- Create an EmailVerificationToken: generate a secure random token (could use `Guid` or crypto random) and expiration timestamp (say 24 hours). Save it via `EmailVerificationTokenRepository.Add(token)`. Possibly also raise an event or directly invoke an EmailService to send the verification email containing the token. (Since we have no external frameworks, we might not implement actual email sending here, but design for it).
- Return success (maybe instruct the caller that verification is required).
- Audit log the registration event.

This process shows how multiple aggregates (User and the Token) and a domain event (send email) come together. Doing it in an application service ensures the database operations (in this case adding user and token) can be done in one unit-of-work (transaction). We ensure either both insert or none (to avoid a token without a user or vice versa).

- **Role/Group Management:** Likely, separate application services for administrative functions:
  - Creating a new Role (just call `RoleRepository.Add` after validation).
  - Creating a new Group (`GroupRepository.Add`).
  - Assigning a user to a role: We could either have `UserService.AssignRole(userId, roleId)`. Implementation: load user (as aggregate), maybe load role to verify existence, call `user.AddRole(roleId)` (domain method to add to internal list), then save user via repository (which updates the link table). Alternatively, call `RoleRepository` to add link – but better to go through User to maintain integrity (the User aggregate might ensure no duplicate roles, etc.).
  - Assigning a user to a group: similarly, we can have `UserService.AddUserToGroup(userId, groupId)` which loads both and perhaps does `user.AddGroup(groupId)` (if we model that inside user), or `group.AddMember(userId)`. Either way, repositories update UserGroup table. After adding, maybe you want to recompute the user's effective permissions (though usually that's done on the fly by combining group permissions when needed, not stored).
  - Assigning a permission to a group: `GroupService.AssignPermission(groupId, permId)` loads group aggregate, calls `group.AddPermission(permId)`, saves via repository (inserts GroupPermission). Group's invariant logic checks if permId already present, etc.
- These operations ensure the relationships are managed consistently and auditable.
- **Password Reset Flow:** Would involve two parts:
  - Request reset: user provides email, we generate a PasswordResetToken and perhaps send email. Implemented in `AuthService.InitiatePasswordReset(email)`: find user by email, if found, create token (and maybe invalidate any previous tokens for that user), save token, and send email with link (the link contains the token). Audit log it.
  - Perform reset: user clicks link with token and new password: `AuthService.ResetPassword(tokenValue, newPassword)`: find token by value via TokenRepository, ensure not expired; load the associated User; call something like `user.SetPassword(newRawPassword, passwordHasher)` to update the password hash (and perhaps force user to verify password meets criteria again); save user; delete or mark the token as used. Possibly also auto-verify email in case the reset implies they have access to the email. Audit log password change.

- **Email Verification:** `AuthService.VerifyEmail(tokenValue)`: find `EmailVerificationToken`, ensure valid; load user; set user's `IsEmailVerified = true` (maybe also activate status); save user; delete token; maybe reward the user (activate account). Audit log email verified.

All the above domain logic follows **SOLID** principles: - *Single Responsibility*: Each class or module has one job (User entity manages user state and rules, PasswordHasher hashes passwords, TokenService issues tokens, AuthService orchestrates login, etc.). This makes code easier to maintain. - *Open/Closed*: Our design is open for extension. For example, if later we add OAuth login, we won't break existing logic – we'll extend `AuthenticationProvider` usage and perhaps add new services for OAuth flows (but User entity stays intact). If we change hashing algorithm, we can implement a new `IPasswordHasher` without changing the User entity. New permissions or roles can be added via data without code changes. - *Liskov Substitution*: We program to interfaces for repositories and services, so e.g. a `SqlUserRepository` (using Dapper/SQL Server) can be substituted with a `PostgresUserRepository` or an in-memory stub for tests – the higher-level logic doesn't care as long as it fulfills the interface. - *Interface Segregation*: We will define focused interfaces (e.g. `IUserRepository` doesn't force methods unrelated to user persistence; `ITokenService` only concerns token ops). This avoids bloated interfaces. - *Dependency Inversion*: Our high-level policy (domain logic) does not depend on low-level details. Domain and application services depend on abstractions (interfaces) for hashing, token generation, and data access, rather than concrete classes. The infrastructure layer will provide concrete implementations (e.g. Dapper repository, JWT library service) that get injected. This keeps domain logic free of direct dependencies on Dapper or JWT specifics. As Microsoft's docs note, in a Clean Architecture, the infrastructure and implementation details depend on the core, not the other way around <sup>3</sup>.

## Architecture Layers and Project Structure

To implement this design in .NET, we will organize the code into clear **layers**, following a Clean Architecture (onion) approach:

- **Domain Layer (Core):** This is where our **Entities, Value Objects, and Domain Interfaces** live. We define classes for User, Role, Group, etc., along with their behavior (methods). We also define interfaces for repositories (e.g. `IUserRepository`, `IRoleRepository`, etc.), and interfaces for domain services like `IPasswordHasher`, `IJwtTokenGenerator`. These interfaces represent dependencies the domain or application logic will need, but the implementation is not defined here. The domain layer should have **no dependencies on EF, Dapper, or ASP.NET** – it's pure C# logic and interfaces. This layer can be a Class Library project (e.g. **MyAuth.Core.Domain**).
- **Application Layer (Services/Use Cases):** This layer orchestrates use-cases and contains **Application Services** or sometimes Command/Query handlers. Here we implement the methods like `AuthService.Login`, `AuthService.Register`, `UserManagementService.AssignRoleToUser`, etc. These use the domain layer classes and interfaces to perform actions. This layer may also implement some domain service logic that wasn't in entities (like possibly an implementation of a domain service that coordinates multiple aggregates). It can also define **DTOs** or models for transferring data to/from outside (though for a simple project, the Web API layer might directly use domain entities or simple models). Application layer depends on Domain layer. It also doesn't have direct data access code, but it calls repository interfaces which will be supplied by infrastructure. We might put any **business logic that doesn't fit**

**in entities** here. For example, checking a user's permissions for an action could be an application service method that loads user, groups, and evaluates. The Application layer is also a good place for **transaction management** – e.g. beginning and committing a DB transaction around a use-case (although in .NET often this is handled by the EF context or explicit transaction scopes in the infrastructure calls). We ensure that each use-case either fully succeeds or fails (to maintain consistency). This can be a Class Library (e.g. **MyAuth.Application**).

- **Infrastructure Layer:** This contains **technical implementations** for the abstractions. Key components:

- **Data Access:** Dapper-based repository classes, e.g. `SqlUserRepository : IUserRepository`. This will use Dapper to execute SQL queries or stored procedures against a SQL Server database. To keep it flexible for other relational DBs, we will stick to standard SQL and parameters that are not SQL Server-specific (or isolate any DB-specific SQL). If using Dapper, we likely use `IDbConnection` interface. We can use dependency injection to provide a connection (or a factory that gives a connection). Alternatively, each repository can internally create a connection from a connection string, but better to inject a connection or a Dapper context. In .NET, you might register `IDbConnection` with a connection string, or use a small ORM wrapper. Regardless, the repository will map database rows to domain entities. We have to be careful with mapping due to our encapsulation:

- If our domain entities have private constructors or properties (to enforce invariants), the repository might need to use reflection or a mapping library. One strategy (to avoid breaking encapsulation) is using the approach of internal constructors for domain entities and use `InternalsVisibleTo` for the infrastructure assembly <sup>10</sup>. For instance, User's constructor might be internal and accept all fields including hashed password – the repository (in Infrastructure project, marked as friend) can use it to reconstitute a User from the DB. Another approach is to materialize a Dapper query into a dynamic or a DTO, then call a domain factory method to create the entity. For example, we could have `User.Reconstitute(userId, username, emailVO, passwordHashVO, status)` as an internal static method. The repository would get the data, build the Email VO, PasswordHash VO, then call that factory. This ensures any validation (if needed during rehydration) happens and the entity is valid. This way, we maintain the rule that domain objects are always in a valid state <sup>11</sup> <sup>12</sup>.
- Repositories should also implement necessary queries, e.g. `FindByUsername`, `FindByEmail` for User. The Medium DDD example explicitly mentions providing specific queries like `findByEmail` in `UserRepository` for uniqueness check <sup>13</sup>.

- **Security:** Concrete implementation of `IPasswordHasher` (e.g. using BCrypt.Net library or ASP.NET Core's PasswordHasher). And implementation of `IJwtTokenGenerator` using JWT libraries. For JWT, we define how to include claims. Often, we might embed user's roles or permissions in the JWT as claims so that other services can authorize without querying our database. However, including too much can bloat the token; a common practice is to include role names or a user type claim. Since this is a boilerplate, we can include minimal info (user ID, maybe username and role names, plus expiration). The signing key (symmetric key or RSA) should be configured in a secure way (from config or environment). We also define token expiration policy (e.g. access token 15m, refresh token 7d if using refresh).

- **Email/SMS Integration:** If we send verification or reset emails, the infrastructure layer can have an `ISender` implementation using an SMTP server or third-party API. Our application service

would call something like `emailSender.Send(to, subject, body)` when needed. For now, we may not implement but design for it.

- The infrastructure layer also includes any other external integrations, like if we later integrate a 3rd-party identity provider, or logging, etc.

All these are wired up via dependency injection so that the application layer or controllers get the right implementations at runtime.

- **API/Presentation Layer:** This is the ASP.NET Core Web API project (e.g. **MyAuth.WebApi**). It will contain **Controllers** or minimal API endpoints that map to our application services. For example, an `AuthController` with `[POST] /api/auth/login` will call `AuthService.Login(request.Username, request.Password)`, then return the JWT in response. Similarly, `[POST] /api/auth/register` calls `AuthService.Register(...)`. We also secure certain endpoints (like user management) with authorization attributes – but since this is the auth service itself, it might be mostly open for login/registration, and possibly protected endpoints for admin operations (like creating roles requires an admin JWT). We will use JWT Bearer authentication in ASP.NET: i.e. configure JWT bearer middleware with the same signing key so that incoming tokens in `Authorization` header are validated and user identity (`ClaimsPrincipal`) is set. That way, in protected endpoints we can get the user's ID from the token. We might add policies like requiring a certain role claim for admin-only APIs.

The Web layer will reference the Application (and thus Domain) and Infrastructure. It sets up the DI container, e.g. `services.AddScoped<IUserRepository, SqlUserRepository>();` etc., `services.AddSingleton<IPasswordHasher, BcryptPasswordHasher>();`, `services.AddSingleton<IJwtTokenGenerator, JwtTokenService>();`. Also configure authentication:

```
services.AddAuthentication(...).AddJwtBearer(...with key and validation parameters...).
```

**Database Considerations:** We will use **SQL Server** for implementation, but keep the design adaptable to other relational DBs: - Dapper uses plain SQL, so to support another DB, we ensure our SQL is standard. (For example, use ANSI SQL and avoid T-SQL specific features or if used, abstract them). - The connection string can be swapped to a different provider (if using System.Data common interfaces, we might just change the connection type). - Alternatively, one could use an ORM like Entity Framework with provider model, but since we choose Dapper (for lightweight and performance), we stick to writing SQL which is fairly portable. If in future, supporting multiple DBs becomes important, one strategy is to use an **interface for SQL generation** or a micro-ORM that handles multiple providers. However, likely just ensuring no hard-coded SQL Server-only syntax (like `GETDATE()` vs standard `CURRENT_TIMESTAMP`) will suffice.

**SOLID in Architecture:** We've structured layers to enforce separation of concerns (UI vs business logic vs data access), which is SRP at a high level. We use interfaces (e.g. repository interfaces) to invert dependencies (the domain is not coupled to Dapper or EF). This follows the dependency inversion principle and Clean Architecture style <sup>3</sup>.

To visualize the architecture, imagine an onion diagram: at the core is the Domain Model (entities, value objects, domain services interfaces). Around that is Application layer (use-case coordination, still pure business logic, no external concerns). The outer layer is Infrastructure (implementations of interfaces,

Dapper, JWT, email, etc.), and outermost is the Web API (controllers). **Dependencies flow inward** – e.g. controllers call application services, application services use domain and call interfaces, infrastructure implements those interfaces <sup>3</sup>. The domain itself doesn't know about controllers or databases. This makes the system testable and maintainable.

(We can represent a typical clean architecture layering as in Figure below.)

14 15

(Figure: Typical layered architecture with UI, business logic, and data access layers, enforcing one-directional dependency: UI -> BLL -> DAL. In our design, we refine this with Clean Architecture, allowing dependency inversion so that business logic doesn't directly depend on the DAL code.)

## Use Case Workflows in the Domain

Now, let's walk through the main authentication scenarios and how our DDD design handles them:

### 1. User Registration

**Goal:** Create a new user account with username, email, and password. Require email verification before activation.

- **API Call:** `POST /api/auth/register` with `JSON { username, email, password }`.
- **Controller:** Calls `AuthService.Register(requestDto)` in Application layer.
- **Application (AuthService.Register):**
  - Validate input format (could delegate to VOs: attempt `new Email(email)` and `new RawPassword(password)` which enforce rules).
  - Check uniqueness: `userRepo.FindByEmail(email)` and `FindByUsername(username)` to ensure not taken (these return null if not found). If either exists, return error response.
  - Hash password: `hashed = passwordHasher.Hash(password)`, producing a hash (and salt internally).
  - Create User aggregate: `user = new User(UserId newGuid, username, EmailVO(email), PasswordHashVO(hashed), status=Inactive, createdAt=now, etc.)`. Perhaps use a factory method `User.CreateNew(...)` that sets up default status and any other initial values.
  - Save user: `userRepo.Add(user)`. (Since no ID generation issues if using GUID; if using DB auto-increment, `Add` will return the new ID or fill it).
  - Create email token: generate random token string (e.g. 32 bytes base64). `verifyToken = new EmailVerificationToken(tokenId=GUID, userId=user.Id, token=value, expires=now+24h)`.
  - Save token: `tokenRepo.Add(verifyToken)`.
  - (Optional: Immediately commit the transaction here if one is open, so the user is persisted before sending email.)
  - Send verification email: compose a verification URL containing the token. Use an EmailSender service (infrastructure) to send the email to the user's email address. (Since this is a boilerplate, we mention it but actual sending might be configured via SMTP settings. Alternatively, simply log the token or return it in dev mode.)

- Return result: maybe a success message like "Registration successful, please verify your email". We **do not** log the user in yet (common practice is to require verification first, but some systems log in immediately with unverified status – we choose stricter approach here).
- Log event: create an AuditLog entry "User Registered" with userId and IP, via AuditLog repository or service.
- **Domain Logic:** The User entity at creation could enforce certain rules (no empty username, password hash not null, etc.). The Email VO ensures format, Password VO ensures strength. We might mark the new user's `Status = PendingVerification` (custom status) until they verify.
- **Consistency:** By doing uniqueness checks in the app layer with repository queries, we enforce unique email/username. There is a small chance of race condition if two registrations happen simultaneously with same email – to truly enforce, also add a unique constraint in the database for those fields. The repository should propagate a uniqueness violation as exception if it occurs, which we handle.
- **Transaction:** Wrapping the `Add(user)` and `Add(token)` in the same transaction ensures we don't send an email if user wasn't saved. If email sending fails, we could still commit user or decide to retry sending asynchronously – that's a design choice (often, we'd commit user and token, and try email separately, because email failure shouldn't prevent registration in most cases; we can notify user to retry verification email).

## 2. Email Verification

**Goal:** Activate a newly registered user by verifying the email token.

- **API Call:** `GET /api/auth/verify-email?token=XYZ` (or could be a POST). In a REST API, this might be a link the user clicks that our front-end turns into an API call.
- **Controller:** Calls `AuthService.VerifyEmail(token)` in Application layer.
- **Application (AuthService.VerifyEmail):**
- Look up token: `tokenRepo.FindByTokenValue(tokenString)` -> returns EmailVerificationToken entity or null.
- If not found or expired, return error (invalid or expired token).
- If found, get userId from it, then `userRepo.GetById(userId)` to load the User aggregate.
- Set user's email verified status: e.g. call `user.MarkEmailVerified()` domain method. This sets a flag and possibly changes Status from PendingVerification to Active. This method might also ensure this action is only done if it was not verified before (idempotency).
- Save user: `userRepo.Update(user)` (persist status change).
- Delete or mark token used: `tokenRepo.Delete(tokenId)` or mark as consumed (to prevent reuse).
- Possibly issue some reward or next step (some systems log the user in or send welcome notification). Here, we simply activate account.
- Return result: e.g. a simple message "Email verified, you can now login." (Or redirect the user if this was accessed via a web context).
- Audit log: record an event "EmailVerified" with userId.



- **Domain Logic:** The `MarkEmailVerified()` method on `User` could also set `Status = Active` if previously pending, or set a `EmailVerifiedAt` timestamp. If user was locked or inactive for other reasons, we might handle that differently, but for our flow, new users become active after verification.
- **Transaction:** Encompass the load user, update user, delete token in one transaction (assuming they share DB or using distributed transactions if separate – simpler if all in one DB).

### 3. User Login (Authentication)

**Goal:** Verify user credentials and issue a JWT for session. Only active/verified users can login. Also handle if account is locked.

- **API Call:** `POST /api/auth/login` with `{ usernameOrEmail, password }`.
- **Controller:** Calls `AuthService.Login(requestDto)` in Application layer.
- **Application (AuthService.Login):**
  - Identify user: since user can login with username or email, decide how to find them. We might first try by username, if not found, try by email (or the request could explicitly include which, but usually one field is enough and we check both). We can have a single repository method that checks both, or do two lookups. Eg: `user = userRepo.FindByUsernameOrEmail(input)`.
  - If user not found, or if found but status is not Active (e.g. PendingVerification or Locked), then **fail login**. If not active:
    - If PendingVerification, return a specific error "Email not verified".
    - If Locked, return "Account locked. Contact support or try later."
    - If simply not found or wrong, just return generic "invalid credentials" (to not reveal which part was wrong).
    - Optionally, audit log a failed login attempt (with reason if known, e.g. user exists but wrong password or locked).
  - If user found and active, proceed to verify password: `passwordHasher.Verify(inputPassword, user.PasswordHash)`. If false, then:
    - Perhaps increment a failed attempt counter on the user. If we implement account lockout (say lock after 5 failures), this is where we'd update that. This would require saving user's FailedAttempt count. To avoid heavy transaction on each fail, some systems do it in memory or use a cache, but we can update in DB for simplicity. If count >= threshold, set user.Status = Locked. Save user (this means a write on login failure – optional feature).
    - In any case, return "invalid credentials" on wrong password. And log the failure in AuditLog (with maybe IP).
  - If password is correct:
    - Reset failed attempts counter (if we track it) and maybe update last login timestamp on user: `user.RecordSuccessfulLogin(now, ip)` could be a method that sets `LastLoginAt` and clears fail count. Save user (this write isn't strictly necessary but is useful info).
    - Create JWT: call `token = jwtService.CreateToken(user)`. This uses our JWT implementation to create a signed token. We include claims like sub (userId), maybe username or email, and role claims or scope. We also set expiry (e.g. expires in 15 minutes or as configured).

- Create Session record: `session = new Session(id=GUID, userId=user.Id, token=jti, issuedAt=now, expiresAt=now+15m)`. Here, `tokenService.CreateToken` could also return the JWT ID (jti claim) or we can generate one. Alternatively, we could store the whole JWT string, but storing jti and userId is enough for blacklist (since token will carry jti as unique id). Save via `sessionRepo.Add(session)`.
  - Return the JWT (and perhaps also a refresh token if implementing refresh tokens; but we said not yet, perhaps later extension).
  - Audit log: "LoginSuccess" event with userId and IP.
- **Domain Logic:** The `User` entity might have a method `VerifyPassword(rawPassword, hasher)` that returns true/false. But usually the verification is done by hashing the input and comparing to stored hash. We might have kept salt and hash and algorithm info in PasswordHash VO, which has the verify function. We could do `if (!user.PasswordHash.Matches(inputPassword)) ...`. That method internally uses the appropriate hasher (maybe static or injected). We prefer to use the injected service directly in the application layer to avoid putting hashing code in the VO (to keep cryptography centralized). Either approach is fine; a VO with verify can have the algorithm baked in if algorithm is fixed (like Bcrypt outputs self-contained hashes that include salt and cost). That might actually be convenient: for example, Bcrypt hash can be verified without needing separate salt. If using a scheme with separate salt, the salt is stored, so the VO could use it with input. So having `PasswordHashVO.Verify(raw)` is reasonable.
- **JWT creation:** The domain doesn't know about JWT specifics, it just gets a token from service. The token service might need user's roles to include in claims. If so, the AuthService might gather that: e.g. get list of role names via `user.Roles` (IDs) then fetch Role names from RoleRepository, then give to JWT service. Or simpler, we could store role names in token by just including the user's roles from the already loaded user (depending if we loaded them eagerly or lazy). Possibly our `FindByUsername` could join roles, but since using Dapper we can do multi-mapping to get roles in one query if we want. For initial version, including roles in JWT might be optional. One could also just include a generic claim like "role": "Admin" for an admin. Or skip roles in token entirely and rely on permissions via separate check – but other services won't know user's permissions without calling this auth service, so usually JWT carries some authorization info for stateless checks. We'll assume including role names in JWT as claims (or a boolean like isAdmin) for simplicity.
- **Security considerations:** Ensure to use secure password hashing (bcrypt, Argon2, etc.) – never plain or simple hash. Use HTTPS for API calls so password isn't in plain text on the network. JWT should be signed with a strong secret (if symmetric) or key pair. Set appropriate JWT expiration and possibly implement refresh tokens to avoid long-lived tokens (we left refresh for later). Also, consider adding **Multi-Factor Auth** later (not now, but design doesn't preclude it – could integrate via additional domain entity for OTP codes or external service).

## 4. Token Validation & Protected Endpoints

Once a user has a JWT, they include it in `Authorization: Bearer <token>` header to access protected APIs (either in this auth service for user management endpoints or in other services).

- In our **Authentication Service** itself, we'll have admin endpoints like managing roles, etc., which should require an **admin user**. We can enforce that by checking the JWT's claims. ASP.NET JWT bearer middleware will validate the token signature and expiration automatically. It will produce a `ClaimsPrincipal` for the user. We configure it to map JWT claims (like `sub` -> `User.Identity.Name` or similar). We can then use `[Authorize]` attributes:
  - For example, `[Authorize]` on general "get my profile" endpoint (so user must be logged in).
  - `[Authorize(Roles="Admin")]` on an admin-only endpoint (which relies on the JWT having a `"role": "Admin"` claim or similar). Because we have our own roles, we might populate the JWT "role" claims accordingly (like multiple role claims for each role the user has). Then `[Authorize(Roles="Admin")]` works out of the box by checking those claims.
- Alternatively, we could implement a custom policy that checks a permission claim. But since we have roles, we'll use role claims for now.
- **Token Blacklisting/Revocation:** We stored sessions in DB. We can use this to invalidate tokens. For example, if a user logs out or an admin wants to log out a user, we can delete that Session or mark it invalid. However, JWT by default once issued will be accepted until expiry unless we check against a blacklist. To implement logout:
  - Provide an endpoint `POST /api/auth/logout` (authorized). It gets the current user's token (from context) or an input token ID, and then removes that session from DB.
  - The tricky part: making the JWT immediately invalid. One way is to check incoming tokens against the session store on each request (e.g., a custom middleware to verify the token's `jti` is in the DB and not revoked). That means every API call will hit the DB which isn't ideal. Alternatively, use short-lived tokens + refresh (so logout is less relevant because token expires quickly) or use a distributed cache for revoked tokens. For our scope, we might not implement token revocation fully, but it's good to note that our Session entity paves the way for it. If performance is a concern, we might skip checking sessions for each request in initial version.
  - For now, we can decide that logout is mostly for client side (they just drop the token) and for security, we rely on short expiration and perhaps manual admin ability to revoke refresh tokens (if we had those). We will not implement a global JWT blacklist check to keep it simpler, but mention that as future improvement.

## 5. Password Reset

(Deferred as an expansion, but design supports it.)

- **Initiate Reset:** `POST /api/auth/password-reset-request` with `{ email }`. App finds user, if exists, creates `PasswordResetToken`, saves it, and send email with link (including token). The link might be something like `https://yourapp/reset-password?token=XYZ`. We ensure the token is one-time use and short-lived (e.g. 1 hour).
- **Confirm Reset:** `POST /api/auth/reset-password` with `{ token, newPassword }`. App finds token, if valid, loads user, calls `user.SetPassword(newPassword, hasher)` which hashes and sets the new hash (and maybe forces user to change at next login if policy, but likely just updates).

Save user, delete token. Possibly also mark any active sessions of that user as invalid (if a security measure, forcing re-login on password change).

- The user can now login with new password. Audit log the password change.

## 6. Role and Group Management

These are more straightforward admin use cases: - **Create Role:** (if needed at runtime) `POST /api/roles` { name, description }. Ensure only admin can call (Authorize). Service calls RoleRepo to add after checking name uniqueness. Alternatively, roles could be predefined in code for simpler systems. - **Assign Role to User:** e.g. `POST /api/users/{id}/roles` with { roleId }. AuthService or a dedicated UserService does: load user and maybe role to verify, then `user.AddRole(roleId)`, save user (UserRepository would handle inserting UserRole in DB). Or directly call a repository method for linking if we keep it thin (but domain method is better to encapsulate any logic like preventing adding duplicate or adding incompatible roles if such rules exist). - **Remove Role from User:** similar, call `user.RemoveRole(roleId)`. - **Create Permission:** (again, possibly predefined rather than dynamic). - **Create Group:** `POST /api/groups` { name, description }. Save new group. - **Assign User to Group:** `POST /api/groups/{groupId}/users` with { userId }. Can be done via user or group as discussed. Perhaps easier: load user, load group to verify existence, then `user.AddGroup(groupId)` and save (insert UserGroup). - **Assign Permission to Group:** `POST /api/groups/{groupId}/permissions` with { permId }. Load group, `group.AddPermission(permId)`, save (insert GroupPermission). - **Remove User from Group / Remove Permission from Group:** analogous.

The design ensures that these operations have places to happen: user aggregate, group aggregate, or application service with repository operations.

### Permission checks during runtime:

When a user calls some other service's API, say they want to access an admin endpoint, that service will trust our JWT's claims. E.g., if an endpoint requires a permission "X", how do we enforce it? Two approaches: 1. Simpler: Map certain roles to permissions in code. For example, say "Admin" role implicitly allows all actions, "User" role allows basic, etc. Then in other services, protect endpoints by role (via JWT claims). This is coarse but easy. 2. Fine-grained: include permission claims in JWT. For instance, if user belongs to a group with permission "VIEW\_REPORTS", then issue JWT with a claim `perm: VIEW_REPORTS`. That way, the reports service can just check for that claim. The downside is if permissions change, the JWT might become stale until next login. Alternatively, other services can call our Auth service to check current permissions (which is more dynamic but adds coupling or network call).

Given our model, one possibility is to compute user's effective permissions at login: fetch all permissions from all groups the user is in, and include them in JWT as a list of perms. This could make the token large if many perms, but might be acceptable if not huge. Or include group IDs and let other service decide group membership (but that other service would then need to know group definitions or call back). Usually, roles suffice for broad separation (admin vs user), and very fine permissions might be handled by internal calls. However, since we designed an elaborate permission system, we may want to leverage it: - We can consider an **Authorization Service** method like `bool UserHasPermission(userId, permName)` that other services could call (or that we could use in the same service for admin decisions). But cross-service calls for every request is heavy – that's what JWT avoids. - Another method: maintain a cache of user permissions. But let's not go too deep; for now, we focus on the auth service itself.

In summary, we will likely add at least role claims to JWT. Permissions might be a future enhancement if needed. This detail can be decided based on requirements of the consuming apps.

## 7. Audit Logging

Every significant action (login success/failure, logout, password change, role assignments, etc.) should produce an AuditLog entry. We can implement this by raising *domain events* in the domain layer and handling them either in application layer or an event dispatcher. For example: - After a successful login, we could raise `UserLoggedInEvent(userId, time, ip)` from the domain service. A handler (possibly in the same service method or a subscribed handler) writes to AuditLogRepository. - Or simpler, just call `auditLogRepo.Add(new AuditLog{ userId, eventType="LOGIN_SUCCESS", timestamp=now, ip=... })` directly in the application service. Given this is a single service deployment, that's fine (no need for pub-sub). Domain events shine in more complex scenarios or distributed systems.

We will likely implement logging inline for now (to keep it understandable). Ensure the AuditLog writing does not break the main flow; if an audit log insert fails, it ideally shouldn't prevent the main operation (we could catch exceptions and maybe log to fallback, but at least it shouldn't crash the login). This might mean we don't wrap audit log insert in the same transaction – depends on how critical logging is. For security, we do want guaranteed logs, but also want the system to still function if log DB is down (maybe queue them). This is beyond scope here, but worth noting.

## Implementation Roadmap

Finally, let's outline the **step-by-step plan** to implement this authentication service following the above design. This acts as a roadmap you can follow:

### 1. Set Up Solution Structure:

2. Create a new .NET Web API project (for example, using `dotnet new webapi -n AuthService`).
3. Add Class Library projects for Domain and Application and Infrastructure. E.g.:
  - `AuthService.Domain` – includes Entities, VOs, repository interfaces.
  - `AuthService.Application` – includes services/use-case classes.
  - `AuthService.Infrastructure` – includes Dapper repositories, implementations of hasher, token service, etc.
  - The Web API project (`AuthService.API`) references Domain, Application, and Infrastructure. Application references Domain, Infrastructure references Domain (for entities, maybe Application for interfaces?), and Domain is self-contained (no outside refs).
4. Configure dependency injection in `Startup.cs` or Program (if using minimal hosting): register services and repositories.

### 5. Model the Domain Entities and Value Objects (Domain Layer):

6. Create the **User** entity class:

- Properties: `Id`, `Username`, `Email` (as Email VO), `PasswordHash` (as PasswordHash VO), `Status` (maybe an enum or VO), `CreatedOn`, etc.

- Collections: e.g. `List<Guid> RoleIds`, `List<Guid> GroupIds`, or possibly actual `ICollection<Role>` if we plan to load them (but that can complicate reconstitution; simpler to store IDs here).
  - Perhaps a collection for external AuthProviders (like a list of AuthProvider VO/entity with fields provider type and external id).
  - Constructors: Make constructor internal (for reconstitution) and a public factory or static method for creating new users (enforcing rules). E.g. `public static User RegisterNew(string username, EmailAddress email, string rawPassword, IPasswordHasher hasher)`. Inside it: hash the password via hasher, create PasswordHash VO, instantiate User with a new ID and provided values, set default status. Validate invariants (non-null username, etc.).
  - Methods:
    - `VerifyPassword(string input, IPasswordHasher hasher)` -> returns bool (could simply call `hasher.Verify(input, this.PasswordHash, this.PasswordHashSalt)`). If using PasswordHash VO with method, could do `PasswordHash.Verify(input)`.
    - `ChangePassword(string newRaw, IPasswordHasher hasher)`: hash new password, set PasswordHash property to new hash VO. Maybe store a `LastPasswordChangeAt`.
    - `MarkEmailVerified()`: set some `IsEmailVerified` bool and if status was Pending, set to Active.
    - `Lock()` and `Unlock()`: change status appropriately (and maybe reset fail count on unlock).
    - `AddRole(Guid roleId)`: if not already in RoleIds, add it.
    - `RemoveRole(Guid roleId)`: remove if exists.
    - `AddGroup(Guid groupId)` / `RemoveGroup(groupId)` similarly.
    - Possibly `AddExternalLogin(AuthProvider provider)` to add an external login link.
    - These methods should enforce needed rules (e.g. not adding duplicate roleId).
    - **Value Objects**: Also define EmailAddress VO (with regex validation in constructor, override Equals/GetHashCode), and PasswordHash VO (with hash string + salt inside, and maybe static `CreateFromRaw(string raw, hasher)` that returns new VO with hashed). The PasswordHash VO's internal constructor might accept the hashed string and salt (called by the static factory or repository). It should be immutable (properties readonly).
7. Define **Role** entity: properties `Id`, `Name`, `Description`. Possibly a method `Rename(string newName)` with validation (ensuring not empty, maybe length limit). Not much behavior unless we add. Ensure uniqueness of name is handled by repository or service on creation.
8. Define **Permission** entity: `Id`, `Name`, `Description`. Possibly treat name as constant key like "CREATE\_USER", etc. Not much behavior either; typically used as data.
9. Define **Group** entity: `Id`, `Name`, `Description`. Also include a collection `PermissionIds` (or even actual Permission objects, but storing as IDs is fine). Methods:
- `AddPermission(Guid permId)`: add to collection if not exists.
  - `RemovePermission(Guid permId)`.
  - Possibly `Rename(newName)` similar to Role.
  - If we decide to track members in Group (which we leaned against to avoid duplication), skip that here. If we did, it'd be a list of UserIds with add/remove similarly.

## 10. Entities for **Session**, **PasswordResetToken**, **EmailVerificationToken**, **AuthProvider**, **AuditLog**:

- **Session**: `Id`, `UserId`, `Token (string or jti GUID)`, `IssuedAt`, `ExpiresAt`. This can be a simple class or even record. Might not need complex behavior, mostly data holder. Perhaps a method `IsActive()` that checks `now < ExpiresAt` (and maybe User's status not locked, but that would be outside domain likely).
- **PasswordResetToken**: `Id`, `UserId`, `Token`, `ExpiresAt`. Possibly `IsExpired()` method.
- **EmailVerificationToken**: similar structure.
- **AuthProvider**: `Id`, `UserId`, `ProviderType`, `ProviderKey` (like external user id). Not much behavior, just linking data. Could be a Value Object without identity since it's tied to user (but since we may need to remove or identify them individually, an `Id` is okay).
- **AuditLog**: `Id`, `UserId?`, `EventType`, `Timestamp`, `Details/Metadata`. This is write-only basically.

## 11. **Repository Interfaces**: Define interfaces in domain or a separate interfaces project (usually domain). For example:

```
interface IUserRepository { User? FindByUsername(string username); User? FindByEmail(string email); User? GetById(Guid id); void Add(User user); void Update(User user); /* maybe Delete, etc. */ }
```

Similarly `IRoleRepository` (`Add`, `GetByName`, `GetById`, `Remove`), `IGroupRepository`, `IPermissionRepository`, `ISessionRepository` (`Add`, `GetByToken`, `Delete`), `IPasswordResetTokenRepository`, `IEmailVerificationTokenRepository`, `IAuditLogRepository`.

The implementations will be in infrastructure. By coding to these interfaces, our application layer can remain unaware of Dapper/SQL specifics.

We might also have a generic `IUnitOfWork` if using transactions explicitly – but since each repository method might internally use a shared DB transaction, we can manage that via DI (e.g. registering a Dapper context or using `TransactionScope`). For simplicity, one could rely on each method doing its own connection and transaction where needed (not ideal for multi-step operations). A better approach: have the application service accept an `IUnitOfWork` or simply use ambient transactions: In .NET, one could use `IDbConnection.BeginTransaction()` and pass the transaction object to multiple repository calls so they use the same transaction. Perhaps our repository methods can accept an optional transaction. Or we can design the infrastructure so that a single Dapper context holds a connection and transaction and is shared via DI (for instance, use per-request scope for connection/transaction). However, careful with concurrency if multiple queries concurrently. Simpler: explicitly manage in service:

- e.g., in `AuthService.Register`: open connection, begin trans, call `userRepo.Add` (passing conn & trans into it), call `tokenRepo.Add` (passing same), commit trans, then send email. This requires our repository implementations to allow receiving an existing connection and transaction. We can implement that or some unit of work pattern. Given space, we assume straightforward use with perhaps simple trans management in code.

## 12. **Domain Services Interfaces**: Define `IPasswordHasher` with `Hash(string password)` -> returns hash (and possibly salt or maybe returns a combined hash string if algorithm includes salt). `Verify(string password, string hash)` returns bool. Or if we use `PasswordHash` VO, then

hasher might only do hashing, and VO's verify does the rest.

Define `IJwtTokenGenerator` with method e.g. `GenerateToken(User user)` -> returns (string token, maybe also returns expiration or jti if needed). Possibly also a method to validate token (but validation is done by ASP.NET pipeline, not needed here explicitly, unless for introspection). We may include `DecodeToken` or so if needed for logout (to get jti or userId). But likely the middleware gives us claims, or we store jti in DB to match.

(We might not strictly need a domain *interface* for JWT generator if we only call it in application and could directly use the JWT library, but defining an interface makes it easier to test and decouple, so we will.)

13. (If any other domain policies or specifications are needed, e.g. password complexity rules could be a separate class or just in hasher config. Keep domain logic central.)

After modeling, unit test your VO's (e.g. that invalid email throws exception, etc.) to ensure validations are correct.

### 1. Implement Application Services (Application Layer):

2. Create classes like `AuthService` (or could name `IdentityService`). This might not be a *literal* service class if using Mediatr or CQRS, but in a simple approach, a class with methods is fine. You can also use the **Command Handler** pattern where each use case has a handler. For clarity, we can just do a few classes grouping related operations:

- `AuthService` for login, logout, register, verify email, reset password.
- `UserService` or maybe just use `AuthService` for everything user-related. But perhaps separate a `UserManagementService` for admin operations (assign roles, etc.), or even separate `RoleService` and `GroupService` if it grows.
- Since it's a small domain, one service class could manage all, but that might violate SRP if it becomes huge. We can at least separate authentication (login/register) from authorization management (role/group admin).

3. In these services, inject the needed repositories and domain services via constructor (DI will provide them from Infrastructure). For example, `AuthService` constructor takes `IUserRepository`, `IPasswordHasher`, `IJwtTokenGenerator`, `IEmailVerificationTokenRepo`, `IPasswordResetTokenRepo`, `ISessionRepository`, `IAuditLogRepository`, ... - that's a lot, maybe too many (indicates perhaps we split responsibilities or group some via a `UnitOfWork` or context). Possibly group some: e.g. a `IIdentityRepository` that encapsulates user, role, group lookups might be created but not necessary. We can inject what we need per method or have the class inject all that it might use.

4. Implement the methods as described in the use case workflows above:

- `Register(username, email, password)`: uses `userRepo`, `passwordHasher`, `tokenRepo`, `emailSender`, etc.
- `Login(usernameOrEmail, password)`: uses `userRepo`, `hasher`, `tokenGenerator`, `sessionRepo`.
- etc.

5. Ensure to handle transactions where needed. For instance, in `Register`, use a transaction for user and token creation. In .NET you could:

- Easiest: If using a single SQL connection, open and use Dapper within a transaction boundary. For example, `using var conn = new SqlConnection(connString);`



```
conn.Open(); using var tx = conn.BeginTransaction(); ... commit; . But our repository methods need to allow passing that tx .
```

- Alternatively, if each repository uses its own connection per call (default), we might temporarily not wrap them (each call separate). That could lead to partial failures (user saved but token not). To avoid, implement transaction properly.
  - Perhaps a simple solution: In AuthService.Register, instead of calling userRepo.Add directly, do the Dapper calls inline because it's a one-off? But that breaks encapsulation of repository. Better to design repository to accept an existing connection/transaction or to utilize an ambient context:
  - We could implement a repository provider that keeps a Connection/Transaction for the scope of a request (using DI scoping). But due to time, we may assume the straightforward approach with manual transaction management in application logic with repositories supporting it.
  - If this detail is too deep, one might assume using an ORM like EF makes transactions easier as SaveChanges covers it. But since specifically we want Dapper, we highlight how to manage it.
6. Add any needed **authorization checks** within these methods if needed (for example, ensure only an admin can call certain operations – but since the controller can enforce [Authorize(Role="Admin")], the service may not need to double-check, unless you want belt and suspenders).
7. These services should return either simple results (booleans, messages) or domain objects that the controllers will map to responses. For example, Login might return the JWT string (and possibly a refresh token). Register might return the new user's ID or a status. We might want to not expose domain entity directly out of service, to keep a clean separation (instead, map to DTO with needed fields). But in an auth service, often after registration you just inform success, you don't send out the whole User object (especially not password). For GetUserProfile, you would map User to a DTO removing sensitive info. This mapping can be done in the application layer or in controllers.

## 8. Infrastructure: Database and Services Implementation

### 9. Database Setup: Design the SQL tables corresponding to our entities:

- Users table (UserID PK, Username, Email, PasswordHash, PasswordSalt, Status, CreatedOn, etc.). Enforce UNIQUE constraints on Username and Email.
- Roles table (RoleID PK, Name UNIQUE, Description).
- Permissions table (PermissionID PK, Name UNIQUE, Desc). Populate with initial permissions if any.
- Groups table (GroupID PK, Name UNIQUE, Desc).
- UserRoles (UserID FK, RoleID FK, PK composite or separate ID).
- UserGroups (UserID FK, GroupID FK).
- GroupPermissions (GroupID FK, PermissionID FK).
- Sessions (SessionID PK (maybe GUID or int), UserID FK, Token (string or jti), IssuedAt, ExpiresAt). If JWT are short-lived, this table can grow quickly; consider a cleanup strategy for expired tokens.
- PasswordResetTokens (TokenID PK or not, could just use the token string as primary key if unique, plus UserID, TokenValue, ExpiresAt).

- EmailVerificationTokens (similar structure). You might combine these into one generic Token table with a type, but separate is fine. Ensure TokenValue is indexed for quick lookup.
- AuditLogs (LogID PK, UserID (nullable), EventType, Timestamp, Metadata (maybe JSON or varchar for IP, etc.)).

Use appropriate data types (e.g. varchars for strings, datetime for timestamps, etc.). If using GUIDs as IDs, use UNIQUEIDENTIFIER in SQL Server.

Write SQL scripts or use migrations (if not using EF, you can manually create tables or use a tool like Flyway or .NET migration without EF). Since not focusing on that, just ensure the schema supports our model.

#### 10. **Dapper Repositories:** For each repository interface, implement it in Infrastructure:

- UserRepository (likely the most complex):
- For `FindByUsername` and `FindByEmail`, a simple SELECT on Users table filtering by username or email. Possibly also join or separate queries to load roles and groups if needed. In early stage, we might not load roles/groups for login (since we only need password and status). But if we want role claims in JWT, we either:
  - do a join query: `SELECT u.*, r.Name as RoleName FROM Users u LEFT JOIN UserRoles ur on u.Id=ur.UserId LEFT JOIN Roles r on ur.RoleId=r.Id WHERE u.Username=@user;` Then map user and fill a list of role names.
  - Or get user first, then get roles via another query (e.g. `SELECT r.Name FROM Roles r JOIN UserRoles ur on r.Id=ur.RoleId WHERE ur.UserId=@id`).
  - Either is fine; Dapper can map multi-mapping but maybe easiest to do two queries. It's okay for now given single user fetch.
- `Add(User user)`: do an INSERT into Users. Use parameters for each field. If using an identity (int) key, capture it (e.g. `SELECT CAST(SCOPE_IDENTITY() as int)` in SQL Server). If using GUID, set user.Id before and just insert. Because we might also want to insert initial roles or groups assignment:
  - However, at registration, user has no roles (maybe a default role like 'User' could be auto-assigned if desired). If yes, we then insert into UserRoles. That might be done in application logic explicitly (like after adding user, call `userRepo.AddRoleAssignment` if we had one). Could also be handled here if we pass roles.
  - But probably keep it simple: new user gets no roles unless explicitly given or a default. We can assign default role in code after insert if needed.
- `Update(User user)`: update fields like Email, PasswordHash, Status, etc. For role changes, since roles and groups are separate tables, maybe have methods in repository for those: e.g. `AddRoleToUser(userId, roleId)`, `RemoveRoleFromUser(...)`, etc. These could be in UserRepository or in RoleRepository depending on design. But one might simply have UserRepository handle all user-affiliated data changes: e.g., implement `AssignRole(userId, roleId)` which does an INSERT to UserRoles. However, having that in repository means a dependency on Role table too – which is fine as it's still within user context. Alternatively, RoleRepository could have a method to add user to role (less intuitive). It's a design call. We might put it in UserRepo for convenience since it's managing user relations. Similarly for groups.
- `FindById(userId)`: SELECT from Users (could also load related if needed).

- Possibly `FindByUsernameOrEmail(string login)` to simplify login lookup. (SQL: where Username=@login OR Email=@login). Ensure indexes exist on those fields.
- RoleRepository:
  - `GetById(id)`, `GetByName(name)`, `Add(Role role)`, `Delete(id)`. These straightforward (INSERT/SELECT).
  - If we want to retrieve roles of a user, we could either do via UserRepo or provide a method here like `GetRolesForUser(userId)` which does the join query to fetch role entities or names. This might not be needed if user repo already covers it or if we embed roles in user queries.
- GroupRepository:
  - `GetById`, `Add`, `Delete`, `AssignPermission(groupId, permId)` (INSERT GroupPermission), `RemovePermission(groupId, permId)`, and possibly `GetPermissions(groupId)` (list perms). Also `GetGroupsForUser(userId)` or `AddUserToGroup(userId, groupId)` (INSERT UserGroup).
  - Similar to roles, the responsibility of linking users could be in UserRepo or GroupRepo. Maybe symmetrical. But to avoid confusion, perhaps:
    - Let UserRepository handle user->role and user->group assignments (because it's adding something to a user).
    - Let GroupRepository handle group->permission assignments (because it's adding to a group).
    - That division seems reasonable.
- PermissionRepository: likely just for lookup or seeding. `GetByName`, `Add` etc.
- SessionRepository:
  - `Add(Session session)`: INSERT.
  - `GetByToken(string token)`: if we decide to check token existence (for logout or validation).
  - `Delete(sessionId or token)`: to revoke.
  - Possibly `DeleteByUser(userId)` to remove all sessions (for when user is locked or password changed).
  - One could also add `DeleteExpired()` periodically if needed.
- Token Repositories (PasswordResetTokenRepo, EmailVerificationTokenRepo):
  - `Add(token)`: INSERT.
  - `GetByToken(string token)`: SELECT where Token = value and not expired.
  - `Delete(tokenId)` or a method to mark used (could just delete).

- Could unify them by having a type, but separate makes it clear.
- AuditLogRepository:
  - `Add(AuditLog log): INSERT`.
  - Possibly queries if we need to fetch logs for audit UI, but not required for auth functionality. If needed, maybe a method to query logs by user or date.

All these repository methods will use Dapper's `connection.Execute` or `connection.Query` as appropriate. We have to handle mapping for multi-table queries: - e.g. in `FindByUsernameOrEmail`, if we join roles, we can use Dapper's multi-mapper to fill a User with a Role list. But that needs writing a custom mapping function. Another approach: query user separately then roles separately. - Simplicity: just get the user row, and then do `GetRolesForUser(id)` and `GetGroupsForUser(id)` to populate user.Roles and user.Groups lists before returning the User entity. That's a couple extra queries but not a huge deal for one user. We can optimize later if needed. This also keeps mapping simple: Dapper can map the user easily to our User entity if columns match property names (with maybe some config for VOs). - But if properties are objects (Email VO), Dapper won't directly map to that. We might do: SELECT email as EmailAddress (if EmailAddress has implicit cast or something, not likely). Instead, we can retrieve string and pass it to Email VO constructor in code. - So repository might do:

```
var data = conn.QuerySingle("SELECT ... Username, Email, PasswordHash,
PasswordSalt, Status ... FROM Users WHERE Username=@u", new {u =
username});
var emailVO = new EmailAddress(data.Email);
var pwdVO = new PasswordHash(data.PasswordHash, data.PasswordSalt); //
assuming we store salt separately. If we use a combined hash (e.g. BCrypt),
we may not need salt field at all, just store the hash.
var user = new User(data.UserId, data.Username, emailVO, pwdVO,
data.Status, data.CreatedOn, ...);
```

Because constructor is internal, we might have to construct via a static or internal factory. Possibly mark the repository class as `[InternalsVisibleTo]` in the Domain assembly so it can call internal User constructor. Or simpler, if using a builder as that SO answer did <sup>16</sup>, but that's more code. We can also temporarily make properties settable internally and use Dapper's dynamic mapping, but better control to avoid partially constructed user.

- Continue with adding roles: do a query for roles:
  - `SELECT RoleId FROM UserRoles WHERE UserId=@id` then maybe get roles by those IDs or join to get names. Or one liner: `SELECT r.RoleId, r.Name FROM Roles r JOIN UserRoles ur ON ur.RoleId=r.RoleId WHERE ur.UserId=@id`. Map that list of (id,name) and then assign to user.Roles (maybe as list of Role entities or just store IDs in user aggregate). If domain user only stores role IDs, maybe we set user.RoleIds = list of ids, and leave it to app logic or another call to get names when needed. For token creation, the service could call RoleRepo to get names for those ids.

11. **PasswordHasher Implementation:** Use a proven library or write a simple one. For example, use BCrypt:

- Install BCrypt.Net (if allowed) or use ASP.NET Core's password hasher (Microsoft.AspNetCore.Identity has a PasswordHasher<T> class we could use without full Identity).
- Alternatively, implement PBKDF2 via Rfc2898DeriveBytes. Given focus on design, we can choose a simple approach:
- For learning, possibly use `BCrypt.Net.BCrypt.HashPassword(password)` and `.Verify(password, hash)`.
- That returns a combined hash string that includes salt and cost. In that case, we only need to store that one string (no separate salt column). That simplifies PasswordHash VO to just one string property.
- We'll do that: store `PasswordHash` as `varchar(60)` for BCrypt. So PasswordHash VO just holds that string, and verify uses BCrypt to compare.
- If we want algorithm flexibility, we might store a version or type too, but not needed now.
- Our `IPasswordHasher.Hash(pwd)` returns the hash string (with salt embedded). `Verify(pwd, hash)` uses BCrypt's verify.

12. **JwtTokenGenerator Implementation:** We can use `System.IdentityModel.Tokens.Jwt` and `Microsoft.IdentityModel.Tokens`.

- We'll need a signing key (say, a symmetric key from config).
- Implementation:

```
var handler = new JwtSecurityTokenHandler();
var key = Encoding.UTF8.GetBytes(_config.JwtSecret); // from config
var descriptor = new SecurityTokenDescriptor {
    Subject = new ClaimsIdentity(new[] {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email.Value),
        new Claim(JwtRegisteredClaimNames.Jti,
            Guid.NewGuid().ToString()) // unique token id
    }),
    Expires = DateTime.UtcNow.AddMinutes(15),
    SigningCredentials = new SigningCredentials(new
        SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256Signature)
};
// Add role claims:
foreach(var roleName in roleNames) {
    descriptor.Subject.AddClaim(new Claim(ClaimTypes.Role,
        roleName));
}
var token = handler.CreateToken(descriptor);
string tokenString = handler.WriteToken(token);
return tokenString;
```

This is one way using the token handler. Alternatively, manually craft `JwtSecurityToken` object.

- We should include `iat` issued time as well automatically by handler or add claim. The `iti` claim is important for tracking.
- We also decide on the `audience` and `issuer` if needed (not strictly needed within one system, but good practice to set them and validate).
- The JWT service might need config for issuer, audience, key, expiry – inject those via options or configuration classes. The Infrastructure can read from appsettings (the Web layer passes config to it).
- Provide a method to validate token if needed (not really, since ASP.NET will do that, but if we have a logout that needs to parse token to get `iti`, we can have a method `GetJti(token)` by decoding it without validating signature perhaps because we trust our own token or use the handler to read it).
- Also include in JWT perhaps a claim for `IsEmailVerified` if one wanted, but not necessary if we block unverified from getting a token at all.

**13. Email Sender:** If we want to simulate or implement actual email sending:

- For a real implementation, configure SMTP or SendGrid etc.
- For now, maybe just log the token or print to console. But since we want this boilerplate to be realistic, perhaps integrate something simple or instruct user to implement.
- We can define `ISender` interface and a dummy `SmtpEmailSender` that picks settings from config. But adding actual email might be beyond scope. Still, we'll leave the hooks.

**14. Configuration:** In Infrastructure, likely we need classes to hold config like `JwtSecret`, etc. You can use `IOptions<JwtOptions>` in the `JwtTokenService` class. For now, maybe directly read from a config provider if simpler.

**15. Logging:** Use standard logging for operational logs (not audit, but to log errors, info).

- E.g. use `ILogger<AuthService>` in application service to log when things happen (like "User X logged in from IP Y").

**16. Web API Layer (Controllers):**

**17. Configure in `Startup`:**

- `services.AddScoped<IUserRepository, UserRepository>();` and similarly for others.
- `services.AddSingleton<IPasswordHasher, BcryptPasswordHasher>();` (singleton fine as it has no state).
- `services.AddSingleton<IJwtTokenGenerator, JwtTokenService>();` or Scoped if it uses per-request data (likely stateless so singleton is okay).
- `services.AddScoped<AuthService>();` and if separate, `services.AddScoped<UserManagementService>();` etc.

- `services.AddAuthentication(options => {options.DefaultAuth... = JwtBearer;})... .AddJwtBearer(cfg => { cfg.TokenValidationParameters = new TokenValidationParameters { ValidateIssuerSigningKey=true, IssuerSigningKey = new SymmetricKey(key), ValidateIssuer=false (or set if using), ValidateAudience=false, ClockSkew = ... } });` Also `services.AddAuthorization()` (and add policy if needed).
- **CORS** if our frontends require it, enable accordingly (for development).
- DB Connection: For Dapper, we could register a connection per scope: e.g. `services.AddScoped<IDbConnection>(sp => new SqlConnection(connString));` and possibly `services.AddScoped(sp => sp.GetRequiredService<IDbConnection>().BeginTransaction());`. But managing manually in code might suffice. Alternatively, open and close inside repository methods (which is simpler for a small app, but repeated opening can be not ideal; though connection pooling mitigates cost). A pattern is to use a Dapper extension or ensure each repo gets a config for connection string and opens connection in each call (closing after).
- If using transactions in app service, maybe we manually use connection objects. Possibly it's fine to call repository which opens/closes, for separate queries. If we truly need a multi-step trans (like register), we can open a connection in service, start transaction, then pass that connection & transaction to repository calls. For that, we might need to cast repository to a concrete class that has methods to set connection or accept IDbTransaction in Add method. To keep repository interface simple, one approach: design repository to accept `IDbTransaction` optional param in methods that need to be part of an external transaction (some libraries allow that). We can implement that in method signature or as a property in repository class. But that complicates interface a bit. Could overload or have separate internal usage. If we skip global transaction in v1, we run risk of partial commit if failure, but maybe we handle exceptions to rollback partially (like if token insert fails, we delete user that was inserted? That's messy). Probably better to support trans. Given time, I'd incorporate minimal support: e.g. `UserRepository.Add` could internally use an existing transaction if provided. We'll assume you manage that carefully.

## 18. Define Controllers:

- `AuthController` with routes:
- `POST /auth/register` -> calls `AuthService.Register`. If success, returns 201 created or 200 OK with message. Possibly do not return user details except maybe ID or a generic success.
- `POST /auth/login` -> calls `AuthService.Login`. If success, returns 200 OK with JWT in body (and maybe user info). Typically, response might contain the token and possibly user info (like name, roles) for front-end to use. At least token and expiration.
- `POST /auth/logout` -> `[Authorize]` gets current user's token (from `HttpContext.User` or the Authorization header). We might require token string in body or assume we can get it from context (the JWT middleware usually does not expose the raw token easily, but one can get it via `Request.Headers["Authorization"]`). Alternatively, in logout request, front-end could send the token explicitly (though redundant). For simplicity, we could not even need to call service: since logout in stateless JWT just means client forgets token. But if we maintain session, we call `SessionRepo.Delete` where token jti = that token's jti. We need the jti from token – which we can get if we stored it in claim. If we included jti in JWT, then in controller we can do `var jti =`

```
User.FindFirst(JwtRegisteredClaimNames.Jti)?.Value;
```

`sessionRepo.DeleteByJti(jti);` . That invalidates it server-side. Return 204 NoContent on success. This endpoint is optional – the main effect is client-side removal, but we include it to support server-side session cleanup.

- `GET /auth/verify-email?token=...` -> calls `AuthService.VerifyEmail`. Could also be a POST with token in body for consistency. If success, maybe redirect to a front-end page or return message. If building a pure API, a GET that returns 200 OK "Email verified" might be fine (the front-end could catch that and show user).
- `POST /auth/request-password-reset` and `POST /auth/reset-password` if implementing those now (or leave for later, but we designed them).
- `UserController` for general user management:
- `GET /users/me` [Authorize] -> returns current user profile (look up User by id from token, return dto with username, email, roles, etc. but not password).
- `PUT /users/me` [Authorize] -> allow user to update profile info or change password (though change password could be separate route).
- `POST /users/{id}/change-password` [Authorize] perhaps for admin to force reset or user to change? Possibly just use reset token flow or allow user with old password to change (which is a different flow).
- `GET /users` [Authorize(Roles="Admin")] -> list users (basic info) for admin management.
- ... and other admin operations:
  - `POST /users/{id}/lock` [Authorize(Roles="Admin")] to lock a user (calls `UserService.LockUser`).
  - `POST /users/{id}/unlock` ...
  - We can also create separate `AdminController` or use the same but with admin auth.
- `RolesController` [Authorize(Roles="Admin")] with:
- `POST /roles` (create new role).
- `POST /users/{uid}/roles/{rid}` (assign role).
- `DELETE /users/{uid}/roles/{rid}` (remove role).
- Or combine in `UserController` as sub-routes as shown above.
- `GroupsController` [Admin] with:
- CRUD for groups.
- Assign/remove user to group ( `POST /groups/{gid}/users/{uid}` ).
- Assign/remove permission to group ( `POST /groups/{gid}/permissions/{pid}` ).
- `PermissionsController` [Admin] to create or list perms (if dynamic, or just to list existing).
- `AuditController` [Admin] to query audit logs perhaps (optional).

19. The controllers mainly delegate to the application services and return appropriate HTTP responses (with proper status codes and messages). Use try-catch to convert exceptions to 400/500 as needed, or better, the service returns error types and controller interprets them. For example, if `AuthService.Login` returns null or throws on bad credentials, catch and return 401 Unauthorized or 400.

20. Add model classes for request/response as needed (DTOs). E.g. `LoginRequest { UsernameOrEmail, Password }`, `LoginResponse { Token, Expiry, UserName, Roles }`, etc. Or could directly use the domain types for simple cases. But better decouple in case



domain changes (plus we don't want to expose PasswordHash, etc.). So mapping in controller: domain User to UserDto (id, username, email, roles).

## 21. Testing and Verification:

- 22. Write unit tests for critical domain logic: e.g. verifying that `PasswordHasher` correctly hashes and verifies, that `EmailAddress` rejects bad emails, that `User.AddRole` prevents duplicates, etc.
- 23. Possibly integration tests for repository (with a test database or transaction rollback to verify queries).
- 24. Test the full register-login flow: register a user, verify email (maybe skip actual sending by retrieving token from repo), then try login, ensure JWT is returned and valid. Also test login failure cases (wrong password, not verified, locked).
- 25. Security test: ensure password is hashed in DB, ensure JWT cannot be forged (use proper key), etc.

## 26. SOLID & Best Practice Review:

- 27. Ensure the code structure respects clean boundaries: e.g. no direct SQL in application layer, no business logic in controllers or repository (the repo should not enforce business rules, just data mapping; domain should handle rules).
- 28. Check that adding a new database (e.g. switching to PostgreSQL) would only require changing connection string and maybe minor SQL syntax, but domain logic stays same. If using only Dapper with parameterized queries, likely fine.
- 29. Check that adding a new feature (like support for Google OAuth login) would be straightforward: we'd add a new entity `AuthProvider` (we have it) and perhaps an `ExternalAuthService` to handle OAuth token exchange and then link to a user (creating user if not exists, with an external login). Our design already has `AuthProvider` in place, so implementing it later means building on what's there, not redesigning core. That shows our design is extensible (Open/Closed principle in action).
- 30. Evaluate performance: Dapper is efficient for queries. Using proper indexes (especially on login fields and foreign keys) is important. Our multi-step queries on login (user then roles) are fine, but if needed, we can optimize to one join query. Considering scale: The auth service usually handles frequent calls (every new request is authenticated via JWT but that's stateless; actual login calls are less frequent). Our design should handle a large number of users easily – queries by indexed fields, and JWT validation is done in memory after signature check. The session table could grow; consider archiving or cleaning. Use caching if needed for things like roles/permissions that rarely change – e.g. caching role list so we don't query DB every time to map role IDs to names for token creation. But premature optimization can be avoided; measure first.
- 31. Maintainability: The code is organized by domain concepts (User, Role, etc.), so it's easy for a developer to find where to change logic related to, say, password policy or token lifetime (in one place). We avoided massive God classes – responsibilities are distributed logically.

## 32. Future Extensions (not implemented now but planned):

- 33. Add refresh tokens for longer sessions: e.g. on login, also issue a long-lived refresh token, store it (like Session table or separate), and provide an endpoint to exchange refresh for new JWT. This

requires storing refresh and maybe a revoke on logout. Our Session entity can accommodate refresh tokens too (store refresh in Session and short JWT not stored).

34. Two-factor authentication: we could add an entity for OTP codes or integrate an external 2FA service. Domain could handle generating and validating codes (maybe as a Value Object like `OtpCode`). The login flow in AuthService would then branch if 2FA is enabled for the user: after password check, instead of issuing JWT, mark login as pending 2FA and send code (via SMS or email), then complete login on code verification.
35. API keys or service-to-service auth: possibly add another entity if needed for API clients.
36. Multi-tenancy: if we needed organizations, we might include a TenantID in User, Group, etc. Out of scope but design could incorporate it with minimal changes (just treat tenant as another attribute or part of key).
37. UI: since it's meant as a reusable service, eventually building a simple admin front-end or CLI to manage users might be beneficial, but back-end is main focus here.

By following this roadmap, you will implement an authentication service that is **domain-centric** and aligned with DDD best practices. The result is a **clean, modular architecture** where each part of the code has a clear purpose, and the business logic for authentication is centralized in the domain model. This makes the solution easy to maintain, extend, and reuse across different projects.

## Conclusion

We designed a robust authentication system using Domain-Driven Design principles, with .NET and Dapper. The domain model defines all core concepts (User, Role, Group, Permission, etc.) and their relationships. We used aggregates (User aggregate and Group aggregate primarily) to maintain consistency of changes, and value objects (Email, PasswordHash) to ensure data integrity. We kept the domain logic independent of technology details. Our application layer handles use-case workflows like registration, login, etc., leveraging domain objects and coordinating repositories and external services (hashing, JWT). The infrastructure layer implements data persistence via Dapper and integrates JWT generation and other technical concerns. Throughout, we applied SOLID and clean architecture, making sure the system is modular (e.g. easy to swap the DB or change auth policies) and easy to reason about (each entity encapsulates relevant rules).

This comprehensive design can be implemented step by step. The result will be a **working authentication service** with RESTful APIs and JWT authentication, suitable as a boilerplate for any .NET project that needs user management. By studying and building this, you'll deepen your understanding of both authentication domain problems and DDD techniques in practice, which was a primary goal of this exercise. Enjoy implementing your Auth Service! 17 8

---

1 2 domain driven design - Implementing DDD: users and permissions - Software Engineering Stack Exchange

<https://softwareengineering.stackexchange.com/questions/367659/implementing-ddd-users-and-permissions>

3 4 5 14 15 Common web application architectures - .NET | Microsoft Learn

<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>

6 7 8 9 13 17 Using Domain-Driven Design to to Create Microservice App | by Eugene Zimin | Medium

<https://medium.com/@eugene-zimin/using-domain-driven-design-to-to-create-microservice-app-e234154a3fd1>

10 11 12 16 c# - DDD principles and repositories with Dapper - Stack Overflow  
<https://stackoverflow.com/questions/54492134/ddd-principles-and-repositories-with-dapper>